OpShin Plutus V3 and Beyond — User Feedback Insights

Executive summary

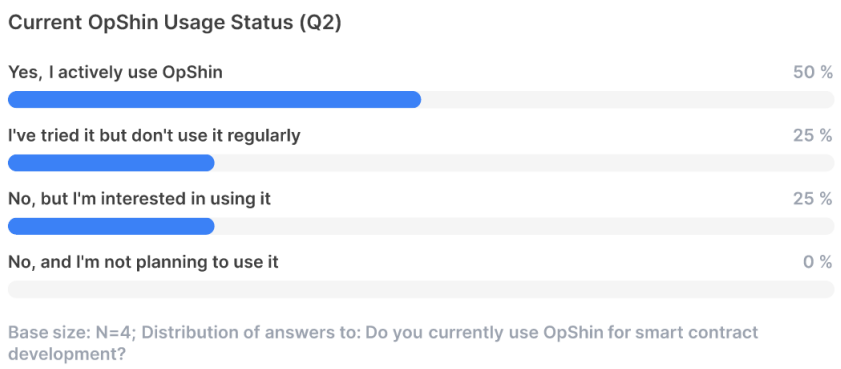
- Study goals: Understand OpShin user experiences and future feature priorities, including **adoption barriers**, **Plutus V3 migration**, **validator design preferences**, and **documentation/usability** needs.
- Key findings:
  - **Documentation** is the standout priority (**100%**), with **Usability (helpful compiler feedback)** also important (**50%**).
  - **Engagement is early**: **75%** have either used OpShin or tried it; many are still in exploration (**25%**).
  - **Decision drivers** center on Python familiarity and usability signals (see Usage and adoption).
  - **Design preferences**: directional signal shows interest in keeping validator(context) while exploring an Aiken-style approach organized "in a class style" (base **4**; see Validator design preferences).
  - **Plutus V3 migration**: No participants answered migration/Blueprints items in this wave; a focused follow-up study/interview wave is recommended.
- **Who we talked to**: **4** participants (mix of developers who have tried OpShin and those interested in it).
- **Bases**: Q2 (**4**), Q3 (**4**), Q4 (**4**); validator preferences Q11–Q12 (**4**). Interpret directionally.

Usage and adoption

Adoption barriers (directional)

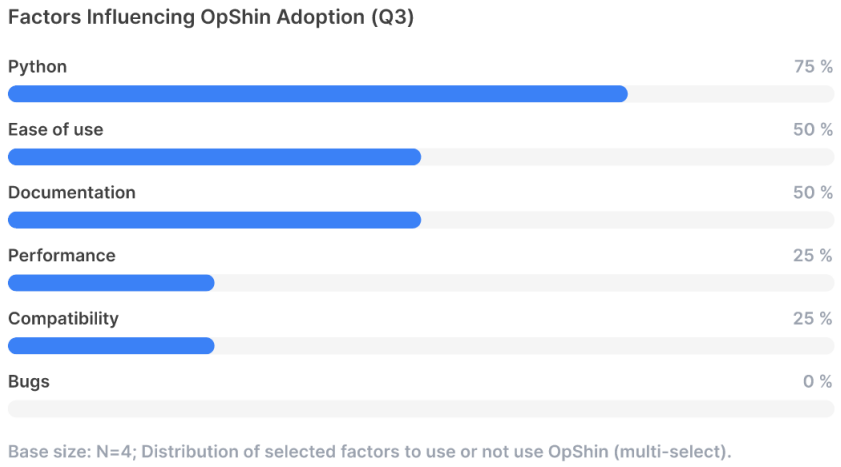
With a small base, barriers align with priorities and drivers: **documentation gaps** and **usability needs** (more helpful compiler feedback). These appear in selection data for Q3 and Q4.

Current status



- Interpretation:
  - **Early-stage engagement**: **75%** have hands-on exposure to OpShin.
  - **Exploratory posture**: Many are still trying rather than using regularly (**25%**), with additional interest from non-users (**25%**).
- Note: With **4** answers to Q2, interpret percentages as directional.

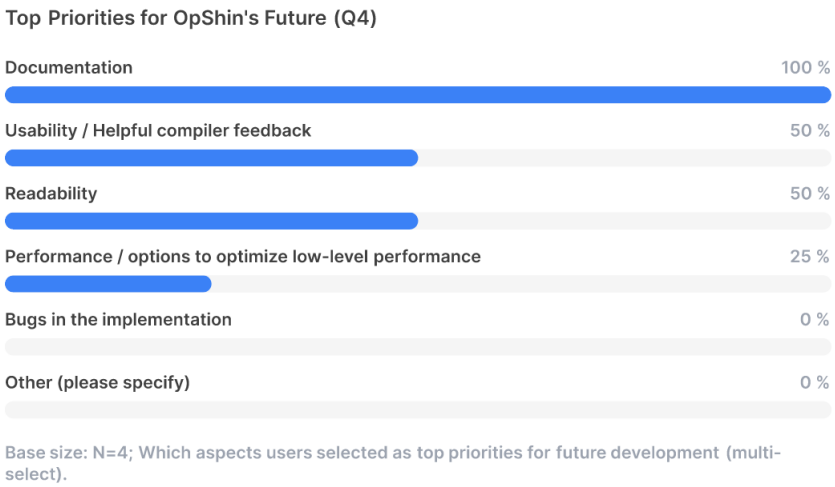
Decision drivers



Python familiarity and **Usability (helpful compiler feedback)** dominate early adoption considerations, reinforcing the need to keep the pythonic developer experience crisp.

Future development priorities

What users selected



- Highlights:
  - **Documentation** leads decisively (**100%**), followed by **Usability (helpful compiler feedback)** (**50%**).
  - Performance, readability, and bug-fixing did not surface strongly in the top-three selections (**25%**, **50%**, **0%**) given this very small base.
- Qualitative color:
  - Users ask for approachable, example-driven learning. Short, practical guides and clearer compiler guidance are recurring themes.

Plutus V3 migration and Blueprints

- We did not capture migration experience in this wave (no participants answered the migration questions). As a result, we cannot quantify V3 usage or Blueprint impacts here.
- Given the importance of these topics, consider a focused follow-up study/interview wave on:
  - Migration readiness checklists and example diffs.
  - Clear guidance on adapting to Blueprint changes, with minimal-ceremony recipes.

Validator design preferences (directional)

- Signals suggest a **balanced posture**:
  - Interest in keeping **validator(context)** for ergonomics.
  - Curiosity toward an **Aiken-style separation** of validators, expressed as “yes, but in a class style,” hinting at structured modularity.
- Supporting quote: "yes, but in a class style"
- Given the base **4**, treat this as exploratory.
- Implications for roadmap:

- Maintain current default, but **offer adapters/aliases** to support explicit datum/redeemer/context and Aiken-like organization patterns.
- Provide side-by-side samples and guidance on when to choose each style to reduce cognitive load.

Recommendations

- **Documentation first**: Expand quickstarts, end-to-end examples, and migration guides; reinforce pythonic ergonomics.
- **Usability next**: Improve compiler feedback with actionable errors, links to docs, and fix suggestions.
- **Design flexibility (pilot)**: Keep validator(context) as the ergonomic default while offering adapters and “class-style” organization for multi-validator contracts; validate this direction in the next wave.
- **Future wave**: Run a focused V3/Blueprints follow-up study/interview wave to quantify migration status and specific blockers; incorporate examples and checklists into the docs.